

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

GA – EXPOSICIÓN SEGUNDO CORTE: HERRAMIENTAS CASE (VISUAL PARADIGM)

MATERIA:

INGENIERÍA EN REQUERIMIENTOS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN, PhD.

EQUIPO G

INTEGRANTES:

FUERTES ARRAES EDSON DANIEL, CI: 0929115087, Correo: efuertes2@uteq.edu.ec

Rol: Líder e integrador – Modelado UML y generación de código

CEDEÑO AVILA WINSTON DAMIAN, CI: 0942833492, Correo: wcedenoa2@uteq.edu.ec

Rol: Marco teórico, descripción del subsistema y conclusiones

CEDEÑO CORONADO WILSON LIZANDRO, CI: 1206867200, Correo: wcedenoc2@uteq.edu.ec

Rol: Justificación de la herramienta MDD

CURSO:

CUARTO SEMESTRE INGENIERÍA EN SOFTWARE “B”

SISTEMA PROYECTO FIN DE CURSO:

MUNDIPETS (DANIEL FUERTES - TITULAR PFC)

Repositorio GitHub:

<https://github.com/MiloSaurio4kHD/PFC-MundiPets-MDD-EquipoG.git>

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

1.	Resumen	3
2.	Marco teórico	3
2.1.	Ingeniería dirigida por modelos (MDE)	3
2.2.	Model-Driven Architecture (MDA): CIM, PIM y PSM	3
2.3.	Desarrollo dirigido por modelos (MDD)	4
2.4.	Transformaciones M2M y M2T	4
2.5.	Metamodelos, perfiles UML, estereotipos y plantillas	4
2.6.	Forward, reverse y round-trip engineering	4
3.	Justificación de la herramienta MDD seleccionada	5
3.1.	Criterios de selección	5
3.2.	El motor Instant Generator	5
3.3.	Comparación con una alternativa descartada: Enterprise Architect	6
3.4.	Aplicación a MundiPets	7
4.	Descripción del subsistema del PFC modelado	7
4.1.	Subsistema seleccionado: gestión del perfil de mascota y validación de información médica	7
4.2.	Actores involucrados	8
4.3.	Requisitos funcionales relevantes	8
4.4.	Alcance incluido y alcance excluido	8
4.5.	Clases de dominio candidatas para el modelo de origen	8
4.6.	Representatividad para la demostración MDD	9
5.	Modelo UML de origen	9
5.1.	Diagrama de clases	9
5.2.	Perfiles y estereotipos aplicados	10
6.	Configuración del mecanismo de generación	11
6.1.	Decisiones de mapeo.	11
6.2.	Plantillas utilizadas y su funcionamiento.	12
7.	Proceso de generación de código	12
8.	Verificación del código generado	14
8.1.	Limitaciones detectadas del generador.	14
9.	Cierre del ciclo (roundtrip) y trazabilidad	15
10.	Reparto del trabajo por integrante	16
11.	Conclusiones	17
12.	Referencias	18
	Anexos	19

Índice de figuras

1.	Diagrama de clases del subsistema en Visual Paradigm.	9
2.	Diagrama de perfil UML con los estereotipos definidos.	10
3.	Configuración del Instant Generator en Visual Paradigm.	11
4.	Ejecución del Instant Generator y árbol de archivos generado.	13
5.	Compilación y ejecución del código generado en el IDE de C#.	15
6.	Modelo modificado con el nuevo atributo en la clase <i>Mascota</i>	16
7.	Código regenerado con el nuevo atributo reflejado.	16
8.	Contribuciones por integrante en GitHub (Insights → Contributors).	17

Índice de tablas

1.	Niveles de abstracción de MDA aplicados a MundiPets.	3
2.	Comparación entre transformaciones M2M y M2T.	4
3.	Actores del subsistema de perfil de mascota y validación médica.	8
4.	Requisitos funcionales núcleo del subsistema.	8
5.	Delimitación del alcance de la demostración.	8
6.	Parámetros de configuración del Instant Generator.	11
7.	Plantillas Velocity utilizadas por el generador.	12
8.	Correspondencia modelo → código (clase <i>Mascota</i>).	14
9.	Reparto del trabajo por tema.	17
10.	Correspondencia completa modelo → código para el subsistema.	19

Índice de códigos de clase

1.	Clase <i>Mascota</i> generada por Instant Generator (fragmento).	14
2.	Clase <i>Usuario</i>	20
3.	Clase <i>Mascota</i>	20
4.	Clase <i>HistorialMedico</i>	21
5.	Clase <i>DocumentoVeterinario</i>	21
6.	Clase <i>CarnetVacunacion</i>	21
7.	Clase <i>Vacuna</i>	22
8.	Clase <i>ValidacionMedica</i>	22
9.	Clase <i>AntecedenteGenetico</i>	22
10.	Clase <i>TipoUsuario</i>	23
11.	Clase <i>EstadoValidacion</i>	23

1 Resumen

Este documento evidencia la aplicación del ciclo completo de desarrollo dirigido por modelos (MDD) sobre el Proyecto Fin de Curso MundiPets, una plataforma orientada a centralizar perfiles de mascotas y apoyar procesos de adopción y cruce responsable. El problema abordado parte de una limitación del proceso actual: la publicación de información incompleta y la falta de datos médicos verificables. Para demostrar el enfoque MDD se seleccionó Visual Paradigm como herramienta CASE, por su cobertura de múltiples lenguajes de salida, la posibilidad de editar sus plantillas de generación mediante Apache Velocity y su edición Community gratuita para uso educativo. El trabajo se delimitó al subsistema de gestión del perfil de mascota y validación de información médica, modelado como un diagrama de clases UML con entidades como Mascota, HistorialMedico, DocumentoVeterinario y CarnetVacunacion, sus asociaciones y operaciones. A partir de este modelo se ejecutó el motor Instant Generator para obtener código fuente, se verificó la correspondencia entre modelo y código, y se realizó un cambio controlado sobre el modelo para evidenciar el roundtrip. El resultado es una base de código estructural trazable, aprovechable como punto de partida para el desarrollo posterior del PFC.

2 Marco teórico

El desarrollo dirigido por modelos reúne un conjunto de enfoques de ingeniería de software en los que los modelos dejan de ser únicamente documentación y pasan a convertirse en artefactos centrales del proceso de construcción. La literatura reciente destaca que el valor práctico de estos enfoques aparece cuando los modelos se vinculan con mecanismos de validación, transformación y generación automática, de forma que sea posible conservar trazabilidad entre decisiones de diseño e implementación [1-3]. En el contexto de MundiPets, este principio permite partir de modelos UML del dominio y utilizarlos como entrada controlada para obtener una base de código reproducible.

2.1 Ingeniería dirigida por modelos (MDE)

Model-Driven Engineering (MDE) es el enfoque más amplio. Considera los modelos, metamodelos y transformaciones como elementos de primera clase durante distintas actividades del ciclo de vida. Su objetivo no se limita a generar código: también puede apoyar análisis, validación, consistencia, interoperabilidad, evolución y trazabilidad. En la práctica, la automatización es una característica clave, porque reduce la repetición de tareas mecánicas y permite trasladar información entre niveles de abstracción de manera sistemática [2-4].

Aplicado a MundiPets, la MDE comprende la cadena completa que conecta los requisitos del sistema con sus modelos de análisis y diseño, el diagrama de clases construido en Visual Paradigm y los artefactos de implementación derivados. Por ejemplo, una entidad del dominio como *Mascota* puede mantenerse relacionada con atributos, asociaciones y operaciones modeladas, de modo que los cambios estructurales puedan rastrearse hasta el código que corresponda.

2.2 Model-Driven Architecture (MDA): CIM, PIM y PSM

Model-Driven Architecture (MDA) es una forma estructurada de aplicar principios dirigidos por modelos mediante la separación de niveles de abstracción. Estudios recientes sobre MDA mantienen la distinción entre Computation Independent Model (CIM), Platform Independent Model (PIM) y Platform Specific Model (PSM), y destacan que las transformaciones entre dichos niveles permiten reducir dependencias tempranas de la tecnología y automatizar parte del paso hacia la implementación [4, 5].

Tabla 1: Niveles de abstracción de MDA aplicados a MundiPets.

Nivel	Propósito	Aplicación a MundiPets
CIM	Describe el problema, objetivos, procesos y necesidades del negocio sin decidir tecnología.	Necesidad de centralizar perfiles de mascotas, adopción/cruce responsable y validación de información sanitaria; requisitos y actores del SRS.
PIM	Describe estructura y comportamiento del software sin comprometerlo con una plataforma concreta.	Modelo UML del subsistema con clases como <i>Mascota</i> , <i>HistorialMedico</i> , <i>DocumentoVeterinario</i> , <i>CarnetVacunacion</i> y <i>ValidacionMedica</i> , con asociaciones y operaciones.
PSM	Añade decisiones tecnológicas necesarias para construir el sistema en una plataforma objetivo.	El mismo modelo enriquecido con convenciones del lenguaje, tipos concretos, paquetes, persistencia y estereotipos/anotaciones según la configuración definitiva del generador.

Para la demostración del equipo, el PIM debe conservar la lógica del dominio de MundiPets sin depender del lenguaje de salida. El PSM se obtiene cuando el modelo incorpora información específica para la generación. Esta separación es importante porque permite explicar con claridad qué información pertenece al problema y cuál fue introducida exclusivamente para materializar el código [5].

2.3 Desarrollo dirigido por modelos (MDD)

Model-Driven Development (MDD) se concentra específicamente en utilizar modelos como principal insumo del desarrollo y en automatizar la obtención de artefactos ejecutables o cercanos a la implementación. En este sentido, puede entenderse como una aplicación orientada al desarrollo dentro del marco más amplio de MDE. La generación automática de código es uno de sus mecanismos más visibles y ha sido estudiada como medio para aumentar productividad, repetibilidad y trazabilidad, aunque su efectividad depende de la calidad del modelo y del generador utilizado [1, 4].

En MundiPets, la demostración MDD no pretende generar la plataforma completa. El propósito es evidenciar un ciclo controlado sobre un subsistema representativo: modelar sus clases en Visual Paradigm, aplicar la configuración de generación, obtener código, verificar la correspondencia modelo-código y ejecutar un cambio que demuestre el roundtrip.

2.4 Transformaciones M2M y M2T

Las transformaciones Model-to-Model (M2M) convierten un modelo origen en otro modelo destino. Dentro del ecosistema OMG, QVT (Query/View/Transformation) constituye la familia de lenguajes de referencia para expresar transformaciones entre modelos conformes a metamodelos. Conceptualmente, una transformación M2M puede utilizarse para refinar un PIM hacia un PSM, incorporando información tecnológica que no estaba presente en el modelo independiente de plataforma [5].

Las transformaciones Model-to-Text (M2T) producen artefactos textuales a partir de un modelo. El caso típico es la generación de código fuente, aunque también pueden generarse archivos de configuración o documentación. MOFM2T representa la especificación OMG para este tipo de transformación basada en plantillas. Investigaciones recientes sobre M2T muestran que las plantillas combinan texto fijo con expresiones que consultan el modelo, y que la regeneración requiere mecanismos para evitar la pérdida de código escrito manualmente, como regiones protegidas o estrategias de sincronización [6].

Tabla 2: Comparación entre transformaciones M2M y M2T.

Tipo	Entrada	Salida	Ejemplo en MundiPets
M2M	Modelo UML/PIM	Modelo refinado/PSM	Agregar detalles tecnológicos, mapeos de persistencia o convenciones necesarias para el generador.
M2T	Modelo/PSM	Código o texto	Generar clases, atributos, métodos y otros archivos del proyecto a partir del modelo configurado en Visual Paradigm.

2.5 Metamodelos, perfiles UML, estereotipos y plantillas

Un metamodelo define los conceptos válidos de un lenguaje de modelado y las relaciones que pueden existir entre ellos; por ello, un modelo UML debe ajustarse a las reglas de su metamodelo. Sobre esta base, los perfiles UML permiten especializar el lenguaje sin crear una notación completamente nueva. Un perfil puede introducir estereotipos, valores etiquetados y restricciones para expresar información adicional que será interpretada por herramientas o generadores.

En una demostración de MundiPets pueden emplearse, cuando la configuración elegida lo requiera, estereotipos como “Entity”, “Service” o “Controller” para diferenciar responsabilidades. Estos estereotipos no deben añadirse como decoración: su utilidad radica en que exista una regla explícita de mapeo hacia el código generado. De manera similar, las plantillas de generación establecen cómo cada elemento del modelo se convierte en texto. La automatización es reproducible únicamente cuando modelo, reglas y plantillas se versionan de forma conjunta [1-3].

2.6 Forward, reverse y round-trip engineering

Forward engineering es el proceso de derivar código u otros artefactos de implementación a partir del modelo. Reverse engineering realiza el recorrido inverso: analiza una implementación existente para reconstruir o actualizar una representación de diseño. Round-trip engineering busca mantener sincronizados ambos lados cuando

existen cambios sucesivos. Este último escenario es el más exigente porque una regeneración puede sobrescribir modificaciones manuales si la herramienta no dispone de una política clara de fusión o protección [6].

Para la evidencia de MundiPets, el ciclo mínimo debe mostrar: modelo inicial → generación de código → verificación → cambio no trivial en el modelo → regeneración → comprobación del cambio. La trazabilidad entre clases, atributos, operaciones y archivos generados permite demostrar que el resultado no es una copia manual, sino una transformación repetible y verificable.

3 Justificación de la herramienta MDD seleccionada

3.1 Criterios de selección

Para representar el ciclo de desarrollo dirigido por modelos en el subsistema de MundiPets, el equipo optó por trabajar con Visual Paradigm como herramienta CASE. La decisión no fue arbitraria: se basó en tres puntos concretos, qué tantos lenguajes de programación cubre su generador, si se pueden ajustar las plantillas con las que produce el código, y cuánto cuesta adoptarla frente a otras opciones parecidas que también sirven para un trabajo de este nivel.

Lo primero, la variedad de lenguajes que el generador soporta, terminó pesando bastante porque, para cuando tocaba preparar la exposición, todavía no se había cerrado del todo qué tecnología se usaría en la implementación final del PFC. Trabajar con una herramienta que admite varios lenguajes de salida evita rehacer el modelado si, ya avanzado el proyecto, el equipo termina programando en algo distinto a lo mostrado durante la demo. Ningún CASE tool es el único con esta ventaja, pero entre las opciones que se revisaron, Visual Paradigm fue de las que más lenguajes ofrecía sin necesidad de pagar licencias adicionales.

El segundo punto, poder editar las plantillas de generación, importaba porque la idea no era simplemente sacar clases vacías del diagrama, sino un código que ya se pareciera, al menos un poco, a cómo el grupo suele escribir sus proyectos. Con un generador cerrado, casi toda la salida termina reescribiéndose a mano para que encaje con el estilo que se busca, y ahí se pierde buena parte del sentido de automatizar nada. Visual Paradigm deja tocar la plantilla directamente, así que ese trabajo de ajuste se hace una sola vez y sirve para todas las generaciones que vengan después.

El último criterio, el costo, tenía que ver con que este es un proyecto de curso y nadie del equipo va a pagar una licencia comercial. Por eso se prefirieron herramientas con versión gratuita o barata que alcanzara para modelar en UML y generar el código que se necesita para la demo, sin quedar atados a una prueba temporal que se venza justo antes de la fecha de exposición. En ese sentido, Visual Paradigm cumple con este criterio porque ofrece una edición Community gratuita para usos no comerciales, entre ellos proyectos educativos y personales [7].

Antes de quedarse comparando solo Visual Paradigm contra Enterprise Architect, el equipo también le echó un ojo, de forma más rápida, a StarUML y a ArgoUML. Ninguna de las dos pasó el primer filtro: StarUML porque su generación de código cubre menos lenguajes de los que se necesitaban, y ArgoUML porque el proyecto parece tener poco movimiento reciente, lo que no da mucha confianza para depender de él en una entrega. Con eso quedaron solo dos candidatas serias, Visual Paradigm y Enterprise Architect, las únicas con un módulo de generación completo como para sostener la demostración.

Los tres criterios, eso sí, no pesaron igual. La cobertura de lenguajes y el costo funcionaron casi como filtros de entrada, cosas que cualquier herramienta tenía que cumplir para seguir en la carrera, mientras que poder personalizar las plantillas terminó siendo el que desempató entre las dos que sí pasaban los primeros dos filtros. Y ahí es donde Visual Paradigm le ganó a Enterprise Architect: ambas cumplían razonablemente bien lo primero, pero se notaba más diferencia en el tercer punto.

3.2 El motor Instant Generator

Dentro de Visual Paradigm existe un módulo llamado Instant Generator que se encarga de convertir un diagrama de clases UML en código. Puede generar, entre otros, Java, C#, C++, Python, PHP, VB.NET y plantillas pensadas para Hibernate, y también hace el camino contrario: reconstruir el modelo UML a partir de código que ya existe [8]. Esa cobertura amplia le sirve a MundiPets porque el subsistema modelado se puede generar en el lenguaje que finalmente se defina para el PFC sin tener que cambiar de herramienta más adelante.

Otra cosa que pesó en la elección es que Instant Generator no depende de un motor cerrado para producir el código: sus plantillas están hechas sobre Apache Velocity, que es un motor de plantillas abierto y editable [9].

Apache Velocity, de hecho, es un proyecto independiente de código abierto de la Apache Software Foundation, lo que explica por qué sus plantillas pueden modificarse libremente en vez de quedar cerradas dentro del generador [10]. Gracias a eso se puede ajustar cómo quedan formateadas las clases generadas, dónde van las llaves o si se agregan comentarios de cabecera, algo que sirve si el equipo necesita que el código generado siga alguna convención particular del PFC.

En la práctica, para generar código hace falta tener listo el diagrama de clases dentro de Visual Paradigm. Desde ahí se va a Tools > Code > Instant Generator, se elige el lenguaje de salida, se indica dónde debe quedar el código y se marcan las clases del subsistema que se quiere traducir. El motor recorre cada clase y convierte los atributos en campos o propiedades según el lenguaje elegido, las operaciones en métodos vacíos que el equipo tiene que completar después, y las relaciones (asociación, agregación, composición, herencia) en su equivalente dentro del lenguaje, por ejemplo, colecciones cuando la multiplicidad es uno a muchos, o clases hijas cuando hay generalización.

Como las plantillas están sobre Velocity, cada parte del archivo generado (encabezado, declaración de la clase, getters y setters, comentarios) es un fragmento independiente que se puede modificar sin tocar el resto del generador. Eso quiere decir que si el equipo quiere, por ejemplo, que cada clase salga con un comentario indicando quién es el responsable, o con alguna anotación de persistencia en los atributos, ese cambio se hace una vez en la plantilla y queda aplicado para todas las generaciones futuras, sin repetirlo clase por clase.

El módulo también funciona al revés: puede tomar código ya escrito y reconstruir el diagrama de clases correspondiente. Esa función no se va a usar en la demo de MundiPets, pero se tuvo presente al elegir la herramienta como una opción de respaldo, por si en una etapa posterior del PFC hace falta documentar en UML algo que ya se empezó a programar directamente.

Para dar un ejemplo concreto del subsistema, la clase `HistorialMedico`, con atributos como la fecha de creación, las observaciones generales y la referencia a la `Mascota`, quedaría generada con esos mismos campos ya declarados según la sintaxis del lenguaje elegido, más sus métodos de acceso hechos automáticamente. La relación uno a uno con `Mascota` aparecería como una simple referencia a un objeto `Mascota` dentro de la clase generada, y una operación que el equipo haya puesto en el diagrama, como registrar una nueva entrada en el historial, saldría como un método sin cuerpo, listo para que se le agregue la lógica durante el desarrollo del PFC.

También se revisó cómo queda organizada la salida. El generador crea un archivo por clase y respeta la estructura de paquetes o espacios de nombres definida en el propio modelo; si el equipo agrupó las clases del subsistema bajo un paquete, digamos, gestión de perfil médico, esa agrupación se refleja igual en la carpeta generada, en vez de tirar todos los archivos sueltos en un mismo lugar. Esto evita tener que reordenar manualmente los archivos antes de meterlos al proyecto del PFC.

3.3 Comparación con una alternativa descartada: Enterprise Architect

Como alternativa se revisó Enterprise Architect, de Sparx Systems, una herramienta CASE bastante usada para modelado empresarial y arquitectura de sistemas. También soporta generación e ingeniería inversa de código, y tiene buena fama en el modelado de arquitecturas complejas y proyectos grandes.

Aun así, se descartó por dos razones principales. La primera es que su curva de aprendizaje es más pronunciada que la de Visual Paradigm, porque está pensada para arquitectos empresariales con flujos de trabajo más largos, algo que no calza con el tiempo disponible para una exposición de corte parcial. La segunda es que configurar la generación de código en Enterprise Architect no resulta tan intuitivo para alguien que recién empieza, mientras que en Visual Paradigm todo el flujo (Tools > Code > Instant Generator) permite elegir lenguaje, ruta de salida y clases desde una sola ventana, lo que hace más simple la demostración en vivo prevista para el cierre.

Más allá de la curva de aprendizaje y la configuración, también se comparó la documentación disponible para quien recién usa la herramienta. Visual Paradigm tiene guías paso a paso específicas para Instant Generator, con capturas del proceso completo, lo cual ahorró tiempo al equipo durante la preparación. La de Enterprise Architect, en cambio, viene mezclada con la documentación de otras funciones de arquitectura empresarial, así que encontrar justo la parte de generación de código tomó más tiempo de búsqueda.

Otro punto fue el licenciamiento. Enterprise Architect ofrece una prueba de treinta días para quien no tiene licencia, después de la cual hay que comprar alguna de sus ediciones pagas [11], lo que trae el riesgo de que la prueba se venza antes de la exposición si no se coordinan bien los tiempos. Visual Paradigm, en cambio, tiene una edición gratuita para modelado UML básico, que alcanza para modelar el subsistema de MundiPets y completar el ciclo de generación sin depender de una fecha de vencimiento.

Con todo esto, el equipo decidió que, aunque Enterprise Architect es una herramienta sólida para proyectos más grandes, Visual Paradigm se ajusta mejor al alcance y al tiempo que hay para esta exposición en particular, sin que eso signifique descartarla para etapas más adelante del PFC donde el modelo sea más complejo.

También se revisó qué tan fácil es exportar el diagrama como apoyo para el informe escrito del PFC, aparte del código generado. Visual Paradigm permite exportar los diagramas en formatos de imagen comunes y generar un documento de especificación a partir del modelo, lo que ayuda a mantener coherencia entre lo que se muestra en la exposición y lo que se entrega por escrito. Enterprise Architect tiene algo similar, pero pensado para reportes más largos y con más configuración de la que hace falta para una entrega de corte parcial, así que en este caso se vio como una ventaja menor frente al flujo más directo de Visual Paradigm.

3.4 Aplicación a MundiPets

El equipo limitó la demostración al subsistema de gestión del perfil de mascota y validación de información médica, con entidades como Mascota, HistorialMedico, DocumentoVeterinario, CarnetVacunacion, Vacuna, ValidacionMedica y AntecedenteGenetico. La elección de Visual Paradigm tiene sentido justo con este subsistema porque, al correr Instant Generator sobre su diagrama de clases, se puede mostrar de forma directa cómo cada clase, atributo y operación del modelo termina en código ejecutable, sin escribir el esqueleto de las clases a mano. Esto se ve especialmente bien en relaciones como Mascota–HistorialMedico (uno a uno) o Mascota–DocumentoVeterinario (uno a muchos), donde la manera en que Instant Generator traduce esa cardinalidad es, en sí, parte de lo que se busca mostrar.

La demostración en vivo va a seguir una secuencia corta: primero se muestra el diagrama de clases completo del subsistema en Visual Paradigm, señalando los atributos y operaciones de cada entidad; después se corre Instant Generator sobre ese diagrama, escogiendo el lenguaje y la ruta de salida delante del público; y al final se abre el código generado para mostrar, clase por clase, cómo quedaron los atributos, las operaciones y las relaciones del modelo. La idea de esta secuencia es que la audiencia vaya conectando cada parte del diagrama con su equivalente en el código, en vez de ver el código generado como algo aislado.

En concreto, la relación Mascota–HistorialMedico, que es uno a uno, sirve para mostrar cómo Instant Generator convierte esa cardinalidad en una referencia directa entre las clases generadas, mientras que Mascota–DocumentoVeterinario, que es uno a muchos, termina como una colección dentro de la clase Mascota. Poner ambos casos en la misma demo ayuda a que se note que el motor no genera lo mismo para toda asociación, sino que adapta la estructura según la multiplicidad que tenga cada relación en el modelo. Esa misma lógica se aplicaría después a CarnetVacunacion con Vacuna y a ValidacionMedica con AntecedenteGenetico, aunque estas dos quedan fuera del recorrido principal que se va a mostrar en el cierre.

Sobre ValidacionMedica puntualmente, se decidió no meterla en el recorrido principal porque buena parte de lo que hace esta clase tiene que ver con reglas de negocio, por ejemplo, revisar que un carnet de vacunación corresponda a vacunas puestas dentro del rango de edad esperado según la especie de la mascota, y ese tipo de validación no se ve reflejada en el diagrama de clases como tal, sino que se programa después de generar el código, dentro de los métodos que Instant Generator deja vacíos. Comentar esto durante la exposición ayuda a que quede claro qué resuelve la generación automática y qué sigue dependiendo de que el equipo programe manualmente.

Para la parte en vivo, se repartieron roles concretos: una persona va a manejar el teclado y ejecutar los pasos dentro de Visual Paradigm, y otra va a ir narrando en paralelo qué hace cada paso y por qué se relaciona con lo explicado antes. Esto es para que quien maneja la herramienta no tenga que parar a explicar cada clic, algo que suele alargar de más este tipo de demostraciones.

4 Descripción del subsistema del PFC modelado

4.1 Subsistema seleccionado: gestión del perfil de mascota y validación de información médica

Para la demostración del ciclo MDD se propone delimitar el trabajo al subsistema de gestión del perfil de mascota y validación de información médica. La elección es coherente con el alcance de MundiPets, plataforma orientada a centralizar perfiles de mascotas y apoyar procesos de adopción y cruce responsable. La documentación previa del PFC del equipo (entrega parcial del ERS/SRS de MundiPets) identifica como problemas del proceso actual la publicación de información incompleta y la falta de datos médicos verificables, por lo que este subsistema

concentra información estructural, relaciones entre entidades y operaciones que resultan adecuadas para una demostración de generación de código.

El subsistema es representativo porque combina operaciones de registro, actualización, consulta y validación; además, contiene asociaciones uno-a-uno y uno-a-muchos que permiten observar cómo Visual Paradigm traslada estructuras UML a construcciones del lenguaje objetivo. No se pretende modelar toda la red social, sino aislar una unidad coherente que pueda ser generada, compilada y modificada durante la demostración.

4.2 Actores involucrados

Tabla 3: Actores del subsistema de perfil de mascota y validación médica.

Actor	Participación en el subsistema	Responsabilidad principal
Propietario de mascota	Actor principal de gestión	Registra la mascota, mantiene sus datos e incorpora antecedentes o documentos permitidos.
Refugio	Gestor alternativo del perfil	Puede registrar y mantener perfiles de mascotas bajo su cuidado cuando corresponda al proceso de adopción.
Veterinaria / profesional autorizado	Actor de validación	Revisa información y documentación sanitaria y registra el estado de validación.
Adoptante o interesado en cruce	Actor de consulta	Consulta la información autorizada del perfil antes de tomar decisiones dentro de un proceso.

4.3 Requisitos funcionales relevantes

La delimitación se apoya en los requisitos ya documentados y refinados en el ERS/SRS del equipo y en la práctica de modelado UML previa del PFC. Para evitar ampliar innecesariamente el alcance, se consideran como núcleo los siguientes requisitos:

Tabla 4: Requisitos funcionales núcleo del subsistema.

ID	Requisito	Relación con el subsistema
RF-01	Registrar mascota	Crea la entidad principal y sus datos generales.
RF-02a / RF-02b	Registrar y actualizar historial médico	Mantiene antecedentes clínicos vinculados con la mascota.
RF-03	Consultar perfil completo de una mascota	Recupera la información autorizada del perfil y sus relaciones.
RF-09	Validar la información médica de una mascota	Introduce el proceso de revisión por un actor veterinario autorizado.
RF-14	Gestionar carnet de vacunación	Modela información sanitaria estructurada y su relación con vacunas y controles.
RF-16	Gestionar antecedentes genéticos y parentesco	Añade antecedentes útiles para procesos responsables de cruce.
RF-11	Administrar la privacidad de la información	Se considera requisito transversal para controlar qué datos médicos pueden ser consultados.

La práctica experimental de modelado UML del equipo relaciona estos requisitos con clases del dominio como Mascota, HistorialMedico, DocumentoVeterinario, CarnetVacunacion y otras entidades asociadas, confirmando que el subsistema dispone de suficiente riqueza estructural para construir un diagrama de clases de seis o más clases sin incorporar elementos ajenos al objetivo de la demostración.

4.4 Alcance incluido y alcance excluido

Tabla 5: Delimitación del alcance de la demostración.

Incluido en el subsistema	Fuera del alcance de esta demostración
Registro y actualización de los datos esenciales de la mascota.	Gestión completa de solicitudes de adopción.
Asociación del perfil con historial médico y documentos veterinarios.	Gestión completa de solicitudes de cruce.
Carnet de vacunación y elementos sanitarios relacionados.	Mensajería y conversaciones entre usuarios.
Validación de información/documentación médica.	Motor de inteligencia artificial y algoritmos de compatibilidad.
Antecedentes genéticos y reglas básicas de visibilidad del perfil.	Moderación global, reportes y administración integral de la red social.

4.5 Clases de dominio candidatas para el modelo de origen

Sin sustituir la responsabilidad del Integrante 1 sobre el diagrama definitivo, la documentación previa permite proponer como conjunto coherente de clases para este subsistema: Mascota, Propietario/Usuario,

HistorialMedico, DocumentoVeterinario, CarnetVacunacion, Vacuna, ValidacionMedica y AntecedenteGenetico. Este conjunto supera el mínimo de seis clases exigido por la guía y permite representar cardinalidades, colecciones y operaciones con firmas completas. La denominación final debe coincidir exactamente con el archivo .vpp y con el código que se genere.

4.6 Representatividad para la demostración MDD

El subsistema seleccionado ofrece una demostración clara del enfoque MDD porque la entidad *Mascota* funciona como núcleo y se relaciona con objetos que poseen ciclos y responsabilidades distintas. Un cambio como agregar un atributo al perfil, una operación al historial o una relación con una entidad sanitaria puede observarse primero en UML y luego en el código regenerado. Esto facilita construir la tabla de correspondencia modelo → código y mostrar trazabilidad.

Además, la separación del subsistema evita que la exposición se convierta en una revisión general de MundiPets. El objetivo de la demostración debe permanecer en Visual Paradigm y en el recorrido modelo → generación → verificación → roundtrip, utilizando el dominio de MundiPets únicamente como caso real sobre el cual se evidencia el funcionamiento de la herramienta.

5 Modelo UML de origen

El modelo UML de origen del subsistema se construyó íntegramente en Visual Paradigm y se conserva en el archivo nativo *MundiPets-DiagramaClases.vpp*, ubicado en la carpeta *modelo/* del repositorio. A partir de este modelo se ejecutó posteriormente la generación de código descrita en las secciones siguientes.

5.1 Diagrama de clases

El diagrama de clases constituye el artefacto estructural de origen sobre el cual opera el generador. Modela el subsistema de gestión del perfil de mascota y validación médica con ocho clases del dominio y dos enumeraciones, superando el mínimo de seis clases exigido. La Figura 1 presenta el diagrama completo.

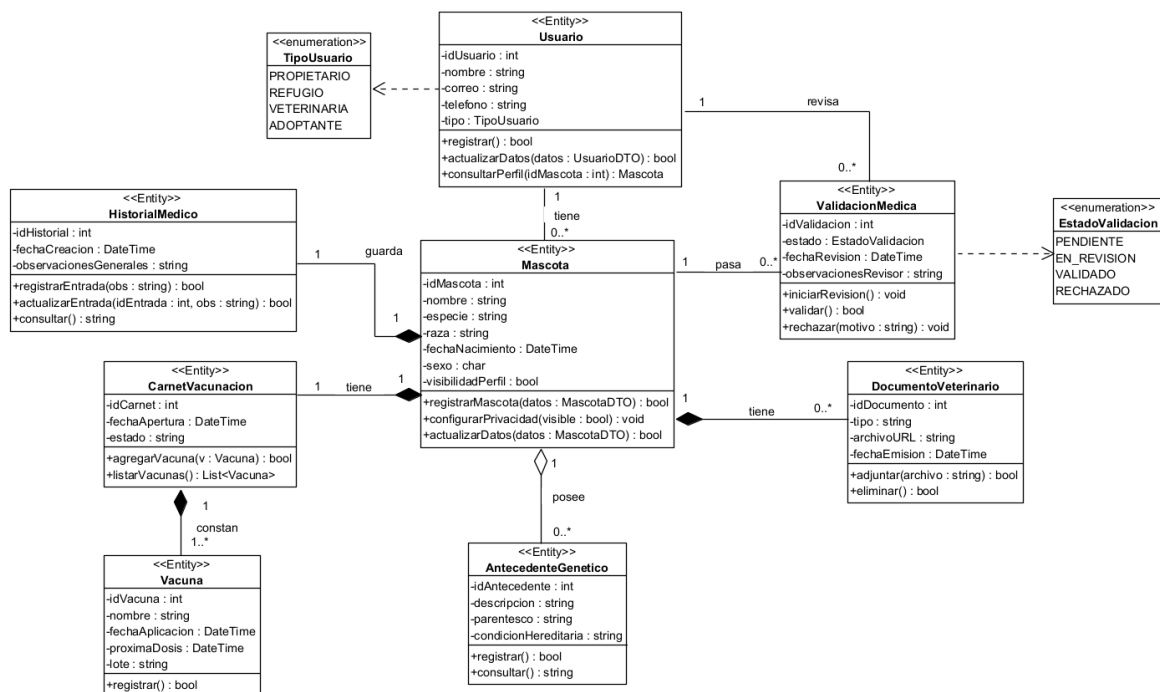


Figura 1: Diagrama de clases del subsistema en Visual Paradigm.

Cada clase se especificó con atributos tipados y visibilidad privada, y operaciones con firma completa y visibilidad pública, siguiendo las convenciones de encapsulamiento. Los tipos de datos se declararon en notación

C# (string, int, bool, char, DateTime) para mantener la coherencia con el lenguaje objetivo de la generación. Las relaciones del modelo cubren la variedad estructural requerida:

- **Composición:** Mascota es el todo que contiene su HistorialMedico (1 a 1), sus DocumentoVeterinario (1 a 0..*) y su CarnetVacunacion (1 a 1); a su vez CarnetVacunacion contiene Vacuna (1 a 1..*). Estas partes no tienen existencia independiente de su contenedor.
- **Agregación:** Mascota agrega AntecedenteGenetico (1 a 0..*), relación más débil en la que el antecedente puede considerarse de forma independiente.
- **Asociación:** Usuario gestiona Mascota (1 a 0..*) y ValidacionMedica vincula a Mascota con el Usuario revisor.
- **Dependencia:** Usuario y ValidacionMedica dependen de las enumeraciones TipoUsuario y EstadoValidacion respectivamente.

El modelo fue verificado con las herramientas de validación de Visual Paradigm, sin errores de sintaxis ni de consistencia que comprometieran la generación.

5.2 Perfiles y estereotipos aplicados

Para comprender el mecanismo de extensión de UML se construyó un perfil propio dentro del proyecto de Visual Paradigm, conservado en `modelo/perfiles/MundiPets-DiagramaPerfil.vpp`. Un perfil UML permite especializar el lenguaje mediante estereotipos que extienden metaclases del metamodelo, junto con valores etiquetados (*tagged values*) que añaden metadatos. La Figura 2 muestra el diagrama de perfil elaborado.

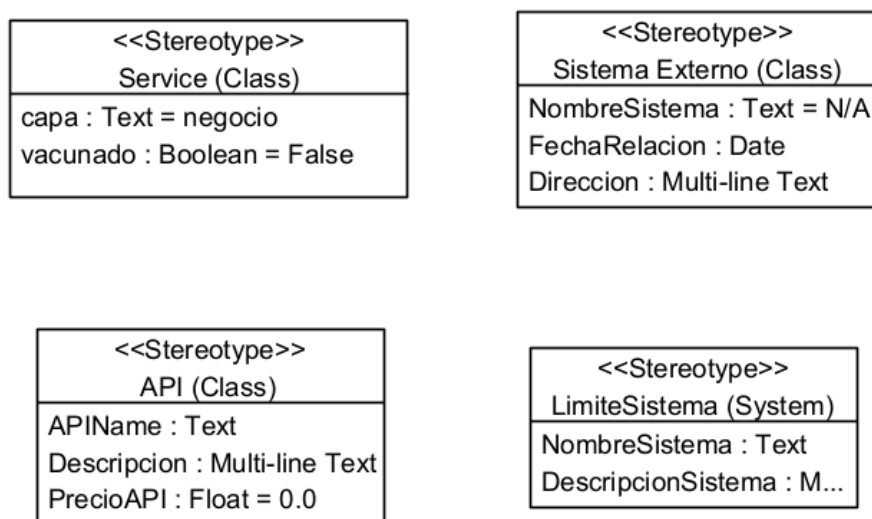


Figura 2: Diagrama de perfil UML con los estereotipos definidos.

El perfil define cuatro estereotipos que ilustran distintas posibilidades de extensión: «Service» y «API» extienden la metaclase Class, «Sistema Externo» extiende también Class, y «LimiteSistema» extiende la metaclase System. Cada estereotipo incorpora valores etiquetados que demuestran cómo un perfil transporta metadatos que UML puro no puede expresar: por ejemplo, «Service» declara los tags capa (de tipo texto) y un indicador booleano, mientras que «API» declara APIName, Descripcion y PrecioAPI. La relación de extensión entre cada estereotipo y su metaclase, indicada entre paréntesis en el diagrama (p.ej. (Class) o (System)), establece a qué tipo de elemento puede aplicarse.

Es importante distinguir este perfil, elaborado como demostración del mecanismo de extensión, del estereotipo efectivamente aplicado en el diagrama de clases del subsistema. Sobre las ocho clases del dominio se aplicó el estereotipo «Entity» (definido a nivel de proyecto), que indica que todas son entidades de datos y que el generador interpreta para producir cada clase con sus campos y métodos de acceso. Los estereotipos del perfil de la Figura 2 no se aplican a las clases del dominio: se mantienen como ejemplo del alcance de un perfil UML para posibles capas o integraciones en fases posteriores del PFC.

6 Configuración del mecanismo de generación

El mecanismo de generación empleado es el *Instant Generator* de Visual Paradigm 17.3, el motor de transformación modelo-a-texto integrado en la herramienta. Instant Generator opera sobre plantillas Apache Velocity (archivos .vm), lo que permite inspeccionar y, de ser necesario, ajustar las reglas con que cada elemento del modelo se convierte en código. Para esta demostración se emplearon las plantillas estándar del lenguaje objetivo, sin personalización, conservadas en la carpeta `plantillas/` del repositorio junto con la captura de la configuración utilizada.

Tabla 6: Parámetros de configuración del Instant Generator.

Parámetro	Valor aplicado
Generador	Instant Generator (Visual Paradigm 17.3)
Lenguaje objetivo	C#
Directorio de plantillas	resources/instantgenerator/csharp (Apache Velocity)
Directorio de salida	generado/ del repositorio
Convención de nombres	Microsoft naming convention (métodos y propiedades en <i>PascalCase</i>)
Propiedades	Auto-Implemented Properties activadas
Política ante regeneración	Sobrescritura con confirmación (<i>Prompt to confirm overwrite file</i>)

La configuración concreta aplicada en la ventana del generador se muestra en la Figura 3.

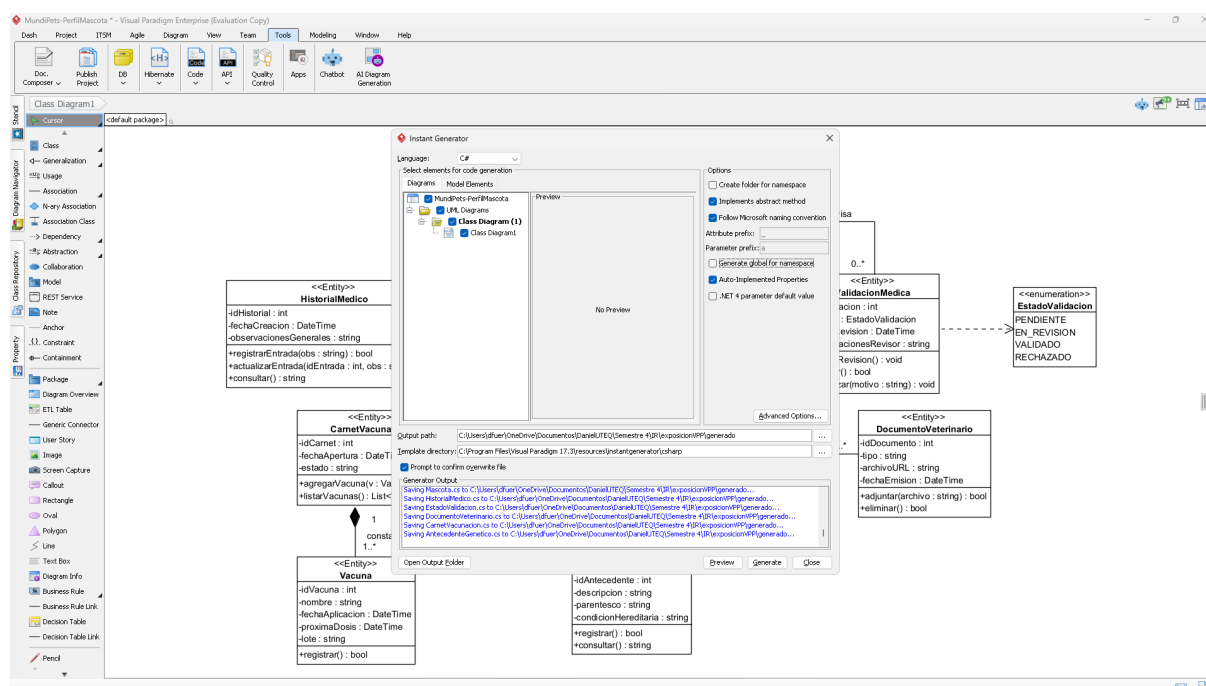


Figura 3: Configuración del Instant Generator en Visual Paradigm.

6.1 Decisiones de mapeo.

A partir de la inspección de las plantillas y del código resultante se identificaron las siguientes reglas de transformación del modelo al lenguaje C#:

- Cada **clase** UML produce un archivo `.cs` independiente con el mismo nombre y la sentencia `using System;`.

- Cada **atributo** privado se traduce en un campo privado con su tipo C# equivalente; los tipos `string`, `int`, `bool`, `char` y `DateTime` se trasladan directamente.
- Cada **operación** se traduce en un método público con su firma completa, cuyo cuerpo se genera como `throw new NotImplementedException()` a la espera de la lógica del desarrollador.
- Los nombres de métodos y propiedades se ajustan a *PascalCase* por la convención de Microsoft (p.ej. `registrarMascota` → `RegistrarMascota`).
- Cada **asociación** genera un campo del tipo de la clase relacionada, tomando como nombre la etiqueta del rol definida en el diagrama; cuando la multiplicidad es de muchos, el campo se genera como **arreglo** de ese tipo (`Tipo[]`).
- Cada **enumeración** produce un archivo `.cs` con la declaración `enum` y sus literales.

6.2 Plantillas utilizadas y su funcionamiento.

El código no se produce de forma monolítica: Instant Generator combina un conjunto de plantillas Velocity, cada una responsable de una parte del archivo generado. Las plantillas empleadas se conservan en la carpeta `plantillas/` del repositorio y se resumen en la Tabla 7.

Tabla 7: Plantillas Velocity utilizadas por el generador.

Plantilla	Responsabilidad
<code>class.vm</code>	Estructura general de la clase: visibilidad, palabra clave <code>class</code> , apertura de llaves y recorrido de atributos, operaciones y asociaciones.
<code>attribute.vm</code>	Genera cada atributo como campo, o como propiedad si están activas las <i>auto-implemented properties</i> .
<code>operation.vm</code>	Genera cada método con su firma y el cuerpo <code>throw new NotImplementedException()</code> .
<code>association.vm</code>	Traduce cada extremo navegable de asociación en un campo del tipo relacionado.
<code>parameter.vm</code>	Genera los parámetros de las operaciones con su tipo y nombre.
<code>enum.vm</code>	Genera la declaración <code>enum</code> y sus literales.
<code>auto-implemented-property.vm</code>	Produce propiedades con <code>{ get; set; }</code> cuando la opción está activada.

El funcionamiento se basa en el mecanismo de plantillas de Apache Velocity: cada archivo `.vm` mezcla texto fijo (que se emite tal cual al código) con directivas de Velocity que consultan el modelo. Por ejemplo, `class.vm` recorre los atributos de la clase con una directiva de iteración (`#foreach`) y, por cada uno, invoca la plantilla `attribute.vm` mediante `#parse`; a su vez, cada plantilla accede a los datos del modelo a través de referencias como `$class.getClassName()` o `$attribute.getVariableType()`. De esta forma, la estructura del código no está escrita a mano en el generador, sino que resulta de aplicar estas plantillas sobre cada elemento del diagrama, lo que explica por qué la salida es reproducible y coherente para todas las clases.

7 Proceso de generación de código

La generación se ejecutó desde el menú `Tools` → `Code` → `Instant Generator`, seleccionando el lenguaje C#, marcando las ocho clases y las dos enumeraciones del diagrama, y estableciendo como directorio de salida la carpeta `generado/` del repositorio. Antes de escribir los archivos se utilizó la función *Preview* para verificar el resultado, y a continuación se ejecutó *Generate*.

El proceso produjo un archivo `.cs` por cada elemento del modelo, sin errores de generación. El árbol de proyecto resultante quedó conformado por los archivos: `Usuario.cs`, `Mascota.cs`, `HistorialMedico.cs`, `DocumentoVeterinario.cs`, `CarnetVacunacion.cs`, `Vacuna.cs`, `ValidacionMedica.cs`, `AntecedenteGenetico.cs`, `TipoUsuario.cs` y `EstadoValidacion.cs`. Las capturas de la invocación, del registro de salida y del árbol de archivos generado se conservan en `evidencias/capturas/`.

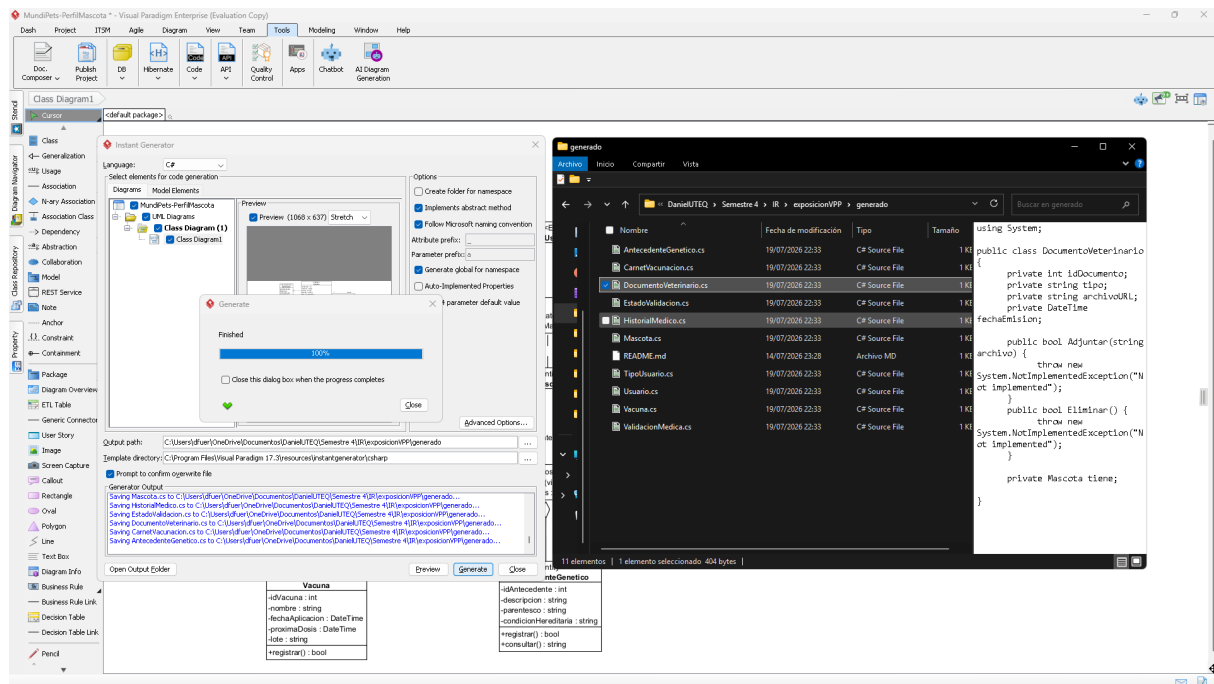


Figura 4: Ejecución del Instant Generator y árbol de archivos generado.

8 Verificación del código generado

El código generado se abrió en el entorno de desarrollo de C# para verificar su estructura y su correspondencia con el modelo de origen. Cada clase del diagrama se materializó en una clase C# con sus campos y métodos, y cada enumeración en su declaración enum, lo que confirma la fidelidad estructural respecto del modelo.

La Tabla 8 resume la correspondencia entre los elementos del modelo y los artefactos generados, tomando la clase Mascota como caso representativo.

Tabla 8: Correspondencia modelo → código (clase Mascota).

Elemento del modelo UML	Artefacto generado en C#
Clase Mascota «Entity»	Archivo Mascota.cs con <code>public class Mascota</code>
Atributo -idMascota : int	<code>Campo private int idMascota;</code>
Atributo -fechaNacimiento : DateTime	<code>Campo private DateTime fechaNacimiento;</code>
Operación +registrarMascota(datos : MascotaDTO) : bool	<code>Método public bool RegistrarMascota(MascotaDTO datos)</code>
Composición con HistorialMedico (1)	<code>Campo private HistorialMedico guarda;</code>
Composición con DocumentoVeterinario (0..*)	<code>Campo private DocumentoVeterinario[] tiene;</code>
Agregación con AntecedenteGenetico (0..*)	<code>Campo private AntecedenteGenetico[] posee;</code>

Un fragmento representativo del código generado para la clase Mascota se muestra en el Código 1.

```

1  using System;
2
3  public class Mascota {
4      private int idMascota;
5      private string nombre;
6      private DateTime fechaNacimiento;
7      private bool visibilidadPerfil;
8
9      public bool RegistrarMascota(MascotaDTO datos) {
10         throw new NotImplementedException("Not implemented");
11     }
12
13     private HistorialMedico guarda;
14     private DocumentoVeterinario[] tiene;
15     private AntecedenteGenetico[] posee;
16 }

```

Código 1: Clase Mascota generada por Instant Generator (fragmento).

8.1 Limitaciones detectadas del generador.

La verificación permitió identificar con claridad qué produce el generador y qué queda a cargo del desarrollador:

- Los **cuerpos de los métodos no se generan**: todas las operaciones se emiten con `throw new NotImplementedException()`, por lo que la lógica de negocio debe programarse manualmente.
- Las asociaciones de multiplicidad múltiple se generan como **arreglos** (`Tipo[]`) y no como colecciones genéricas `List<Tipo>`; si se desea trabajar con listas, es un ajuste manual posterior.
- Los tipos auxiliares empleados en las firmas, como `MascotaDTO` y `UsuarioDTO`, no forman parte del modelo y deben definirse manualmente para que el proyecto compile sin errores.
- El código generado corresponde a la **estructura** del sistema (entidades, campos y firmas), no a su comportamiento; esta es la frontera esperada de la generación estructural a partir de un diagrama de clases.

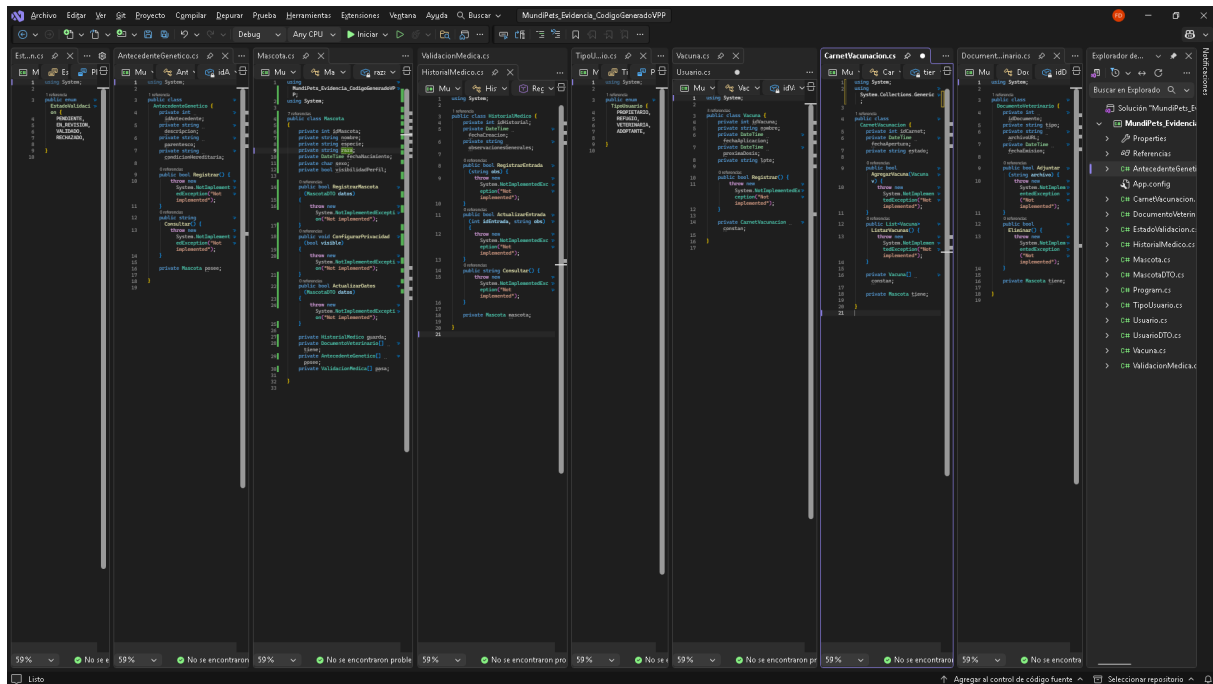


Figura 5: Compilación y ejecución del código generado en el IDE de C#.

9 Cierre del ciclo (roundtrip) y trazabilidad

Para evidenciar el cierre del ciclo se ejecutó un cambio controlado sobre el modelo y se regeneró el código, comprobando que la modificación se reflejara en el proyecto. El cambio consistió en agregar el atributo `-peso : float` a la clase `Mascota` en el diagrama de Visual Paradigm (Figura 6). Tras volver a ejecutar Instant Generator sobre la misma configuración, el archivo `Mascota.cs` incorporó el campo `private float peso;` (Figura 7), demostrando la trazabilidad directa entre una modificación en el modelo y su materialización en el código.

Dado que las plantillas estándar de C# regeneran el archivo completo bajo la política de sobrescritura, la trazabilidad modelo → código es total para la estructura, pero cualquier lógica escrita manualmente en los cuerpos de los métodos se perdería en una regeneración salvo que se preserve mediante control de versiones o bloques protegidos. Esta observación es coherente con lo señalado en el marco teórico sobre los retos del *round-trip engineering* [6].

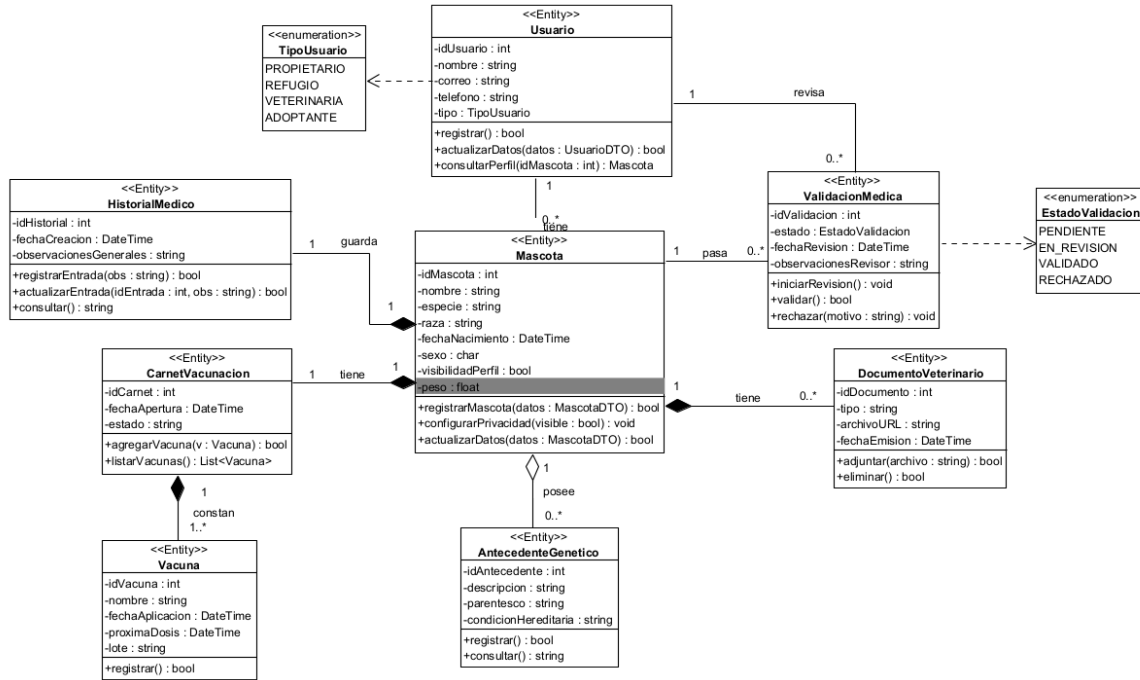


Figura 6: Modelo modificado con el nuevo atributo en la clase Mascota.

```

1 using System;
2
3 public class Mascota {
4     private int idMascota;
5     private string nombre;
6     private string especie;
7     private string raza;
8     private DateTime fechaNacimiento;
9     private char sexo;
10    private bool visibilidadPerfil;
11    private float peso;
12
13    public bool RegistrarMascota(MascotaDTO datos) {
14        throw new System.NotImplementedException("Not implemented");
15    }
16    public void ConfigurarPrivacidad(bool visible) {
17        throw new System.NotImplementedException("Not implemented");
18    }
19    public bool ActualizarDatos(MascotaDTO datos) {
20        throw new System.NotImplementedException("Not implemented");
21    }
22
23    private HistorialMedico guarda;
24    private DocumentoVeterinario[] tiene;
25    private AntecedenteGenetico[] posee;
26    private ValidacionMedica[] pasa;
27
28 }
29

```

Figura 7: Código regenerado con el nuevo atributo reflejado.

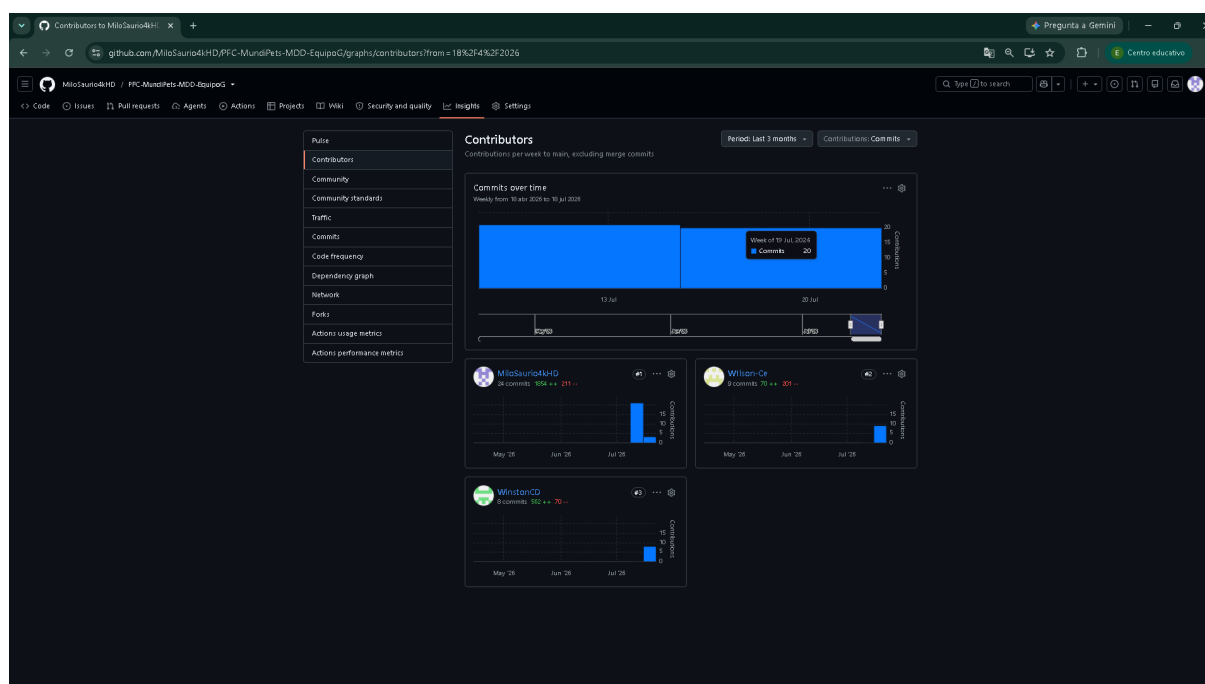
10 Reparto del trabajo por integrante

El trabajo se distribuyó de forma equitativa entre los tres integrantes. La Tabla 9 detalla cada tema del documento y su responsable.

Tabla 9: Reparto del trabajo por tema.

Tema	Desarrollado por
Resumen	Fuertes Arraes Edson Daniel
Marco teórico	Cedeño Avila Winston Damian
Justificación de la herramienta MDD	Cedeño Coronado Wilson Lizandro
Descripción del subsistema del PFC	Cedeño Avila Winston Damian
Modelo UML de origen (diagrama de clases y perfil)	Fuertes Arraes Edson Daniel
Configuración del mecanismo de generación	Fuertes Arraes Edson Daniel
Proceso de generación de código	Fuertes Arraes Edson Daniel
Verificación del código generado	Fuertes Arraes Edson Daniel
Cierre del ciclo (roundtrip)	Fuertes Arraes Edson Daniel
Conclusiones	Cedeño Avila Winston Damian
Modelado en Visual Paradigm y generación de código	Fuertes Arraes Edson Daniel
Integración y compilación del documento LaTeX	Fuertes Arraes Edson Daniel

La evidencia objetiva de esta distribución se encuentra en el historial de *commits* del repositorio GitHub, donde cada integrante contribuyó con su propia identidad Git. El registro de aportaciones puede consultarse en la pestaña Insights → Contributors del repositorio, cuya captura se incluye en evidencias/capturas/.

**Figura 8:** Contribuciones por integrante en GitHub (Insights → Contributors).

11 Conclusiones

El enfoque MDD aporta valor al PFC MundiPets cuando el modelo se utiliza como un artefacto operativo y no únicamente como documentación. La posibilidad de generar una base de código desde un diagrama UML reduce tareas repetitivas, favorece la consistencia estructural y crea una relación verificable entre las decisiones del modelo y los archivos producidos. La literatura reciente señala que la automatización y la trazabilidad son dos de los beneficios más relevantes de los enfoques dirigidos por modelos, siempre que exista disciplina en la gestión de modelos, reglas y artefactos generados [1, 3, 4].

En MundiPets, delimitar la demostración al subsistema de perfil de mascota y validación médica permite comprobar el ciclo sobre una parte real y representativa del PFC sin intentar automatizar el sistema completo. Este subsistema contiene entidades, asociaciones y operaciones suficientes para evidenciar generación, correspondencia modelo-código y un cambio de roundtrip. Por ello, el código generado puede ser aprovechable como línea base estructural para el desarrollo posterior, pero no debe asumirse como una implementación funcional completa: reglas de negocio, seguridad, validaciones específicas, integración y pruebas continuarán requiriendo trabajo de desarrollo.

El principal aprendizaje metodológico es que la calidad de la generación depende directamente de la precisión del modelo y de la configuración del generador. Estereotipos, multiplicidades, tipos y convenciones deben definirse

con intención, porque una inconsistencia del modelo se propaga a los artefactos generados. Del mismo modo, la regeneración exige una política explícita para proteger o separar el código escrito manualmente, ya que el roundtrip es uno de los puntos donde pueden aparecer pérdidas de cambios o divergencias entre modelo e implementación [6].

12 Referencias

- [1] G. Sebastián, J. A. Gallud y R. Tesoriero, “Code generation using Model Driven Architecture: A systematic mapping study,” *Journal of Computer Languages*, vol. 56, pág. 100935, feb. de 2020, ISSN: 2590-1184. DOI: 10.1016/j.cola.2019.100935 dirección: <https://www.sciencedirect.com/science/article/pii/S2590118419300607>
- [2] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi y M. Wimmer, “Low-code development and model-driven engineering: Two sides of the same coin?” *Software and Systems Modeling*, vol. 21, n.º 2, págs. 437-446, abr. de 2022, ISSN: 1619-1374. DOI: 10.1007/s10270-021-00970-2
- [3] I. David, K. Aslam, I. Malavolta y P. Lago, “Collaborative Model-Driven Software Engineering — A systematic survey of practices and needs in industry,” *Journal of Systems and Software*, vol. 199, pág. 111626, 2023, ISSN: 0164-1212. DOI: 10.1016/j.jss.2023.111626 dirección: <https://www.sciencedirect.com/science/article/pii/S0164121223000213>
- [4] M. R. Krouwel, M. Op ’t Land y H. A. Proper, “From enterprise models to low-code applications: mapping DEMO to Mendix; illustrated in the social housing domain,” *Software and Systems Modeling*, vol. 23, n.º 4, págs. 837-864, ago. de 2024, ISSN: 1619-1374. DOI: 10.1007/s10270-024-01156-2
- [5] N. Silega, M. Noguera, Y. I. Rogozov, V. S. Lapshin y T. González, “Transformation From CIM to PIM: a Systematic Mapping,” *IEEE Access*, vol. 10, págs. 90857-90872, 2022, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2022.3201556
- [6] S. Almutairi, A. Zolotas y D. Kolovos, “Towards round-trip engineering of code fragments embedded in models,” en *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, ép. MODELS ’22, New York, NY, USA: Association for Computing Machinery, 2022, págs. 529-538, ISBN: 9781450394673. DOI: 10.1145/3550356.3561578 dirección: <https://doi.org/10.1145/3550356.3561578>
- [7] Visual Paradigm International. “Community Edition — Free UML Modeling Software,” Visual Paradigm, visitado 15 de jul. de 2026. dirección: <https://www.visual-paradigm.com/editions/community/>
- [8] Visual Paradigm International. “UML/Code Generation Software — Instant Generator,” Visual Paradigm, visitado 15 de jul. de 2026. dirección: <https://www.visual-paradigm.com/features/code-engineering-tools/>
- [9] Visual Paradigm International. “Instant Generation in Visual Paradigm — User’s Guide, Part IX, Chapter 2,” Visual Paradigm, visitado 15 de jul. de 2026. dirección: https://www.visual-paradigm.com/support/documents/vpuserguide/276/330_instantgener.html
- [10] The Apache Software Foundation. “Apache Velocity Engine,” Apache, visitado 15 de jul. de 2026. dirección: <https://velocity.apache.org/engine/index.html>
- [11] Sparx Systems. “Enterprise Architect — Free Trial,” Sparx Systems, visitado 15 de jul. de 2026. dirección: <https://sparxsystems.com/products/ea/trial/request.html>

Anexos

Estructura del repositorio GitHub

```
PFC-MundiPets-MDD-EquipoG/
|-- README.md
|-- docs/
|   |-- informe.tex
|   |-- informe.pdf
|   |-- referencias.bib
|   '-- figuras/           (diagramas y capturas del informe)
|-- modelo/
|   |-- MundiPets-DiagramaClases.vpp
|   |-- MundiPets-DiagramaClases.png
|   '-- perfiles/
|       |-- MundiPets-DiagramaPerfil.vpp
|       '-- MundiPets-DiagramaPerfil.png
|-- plantillas/
|   |-- class.vm attribute.vm operation.vm
|   |-- association.vm parameter.vm enum.vm
|   |-- auto-implemented-property.vm
|   '-- ConfiguracionInstantGeneratorMundiPets.png
|-- generado/
|   |-- *.cs               (10 clases generadas)
|   '-- MundiPets_Evidencia_CodigoGeneradoVPP/
|       (solucion C# de Visual Studio)
|-- evidencias/
|   '-- capturas/         (capturas del proceso completo)
|-- exposicion/
|   '-- diapositivas.pdf
|-- borradores/           (aportes individuales por integrante)
```

Tabla de mapeo extensa (modelo → código)

La Tabla 10 presenta la correspondencia completa entre cada clase del modelo y el archivo generado, junto con las asociaciones materializadas en cada uno.

Tabla 10: Correspondencia completa modelo → código para el subsistema.

Clase UML	Archivo generado	Campos de asociación generados
Usuario «Entity»	Usuario.cs	Mascota[] tiene, ValidacionMedica[] revisa
Mascota «Entity»	Mascota.cs	HistorialMedico guarda, DocumentoVeterinario[] tiene, AntecedenteGenetico[] posee, ValidacionMedica[] pasa
HistorialMedico «Entity»	HistorialMedico.cs	Mascota mascota
DocumentoVeterinario «Entity»	DocumentoVeterinario.cs	Mascota tiene
CarnetVacunacion «Entity»	CarnetVacunacion.cs	Vacuna[] constan, Mascota tiene
Vacuna «Entity»	Vacuna.cs	CarnetVacunacion constan
ValidacionMedica «Entity»	ValidacionMedica.cs	Usuario revisa, Mascota pasa
AntecedenteGenetico «Entity»	AntecedenteGenetico.cs	Mascota posee
TipoUsuario (enum)	TipoUsuario.cs	—
EstadoValidacion (enum)	EstadoValidacion.cs	—

Códigos completos de las clases generadas

Se presenta a continuación el código C# completo generado por Instant Generator para las ocho clases del dominio y las dos enumeraciones del subsistema.

```
1  using System;
2
3  public class Usuario {
4      private int idUsuario;
5      private string nombre;
6      private string correo;
7      private string telefono;
8      private TipoUsuario tipo;
9
10     public bool Registrar() {
11         throw new NotImplementedException("Not implemented");
12     }
13     public bool ActualizarDatos(UsuarioDTO datos) {
14         throw new NotImplementedException("Not implemented");
15     }
16     public Mascota ConsultarPerfil(int idMascota) {
17         throw new NotImplementedException("Not implemented");
18     }
19
20     private Mascota[] tiene;
21
22     private ValidacionMedica[] revisa;
23
24 }
```

Código 2: Clase Usuario.

```
1  using System;
2
3  public class Mascota {
4      private int idMascota;
5      private string nombre;
6      private string especie;
7      private string raza;
8      private DateTime fechaNacimiento;
9      private char sexo;
10     private bool visibilidadPerfil;
11
12     public bool RegistrarMascota(MascotaDTO datos) {
13         throw new NotImplementedException("Not implemented");
14     }
15     public void ConfigurarPrivacidad(bool visible) {
16         throw new NotImplementedException("Not implemented");
17     }
18     public bool ActualizarDatos(MascotaDTO datos) {
19         throw new NotImplementedException("Not implemented");
20     }
21
22     private HistorialMedico guarda;
23     private DocumentoVeterinario[] tiene;
24     private AntecedenteGenetico[] posee;
25     private ValidacionMedica[] pasa;
26
27 }
```

Código 3: Clase Mascota.

```
1 using System;
2
3 public class HistorialMedico {
4     private int idHistorial;
5     private DateTime fechaCreacion;
6     private string observacionesGenerales;
7
8     public bool RegistrarEntrada(string obs) {
9         throw new NotImplementedException("Not implemented");
10    }
11    public bool ActualizarEntrada(int idEntrada, string obs) {
12        throw new NotImplementedException("Not implemented");
13    }
14    public string Consultar() {
15        throw new NotImplementedException("Not implemented");
16    }
17
18    private Mascota mascota;
19
20 }
```

Código 4: Clase HistorialMedico.

```
1 using System;
2
3 public class DocumentoVeterinario {
4     private int idDocumento;
5     private string tipo;
6     private string archivoURL;
7     private DateTime fechaEmision;
8
9     public bool Adjuntar(string archivo) {
10        throw new NotImplementedException("Not implemented");
11    }
12    public bool Eliminar() {
13        throw new NotImplementedException("Not implemented");
14    }
15
16    private Mascota tiene;
17
18 }
```

Código 5: Clase DocumentoVeterinario.

```
1 using System;
2
3 public class CarnetVacunacion {
4     private int idCarnet;
5     private DateTime fechaApertura;
6     private string estado;
7
8     public bool AgregarVacuna(Vacuna v) {
9         throw new NotImplementedException("Not implemented");
10    }
11    public List<Vacuna> ListarVacunas() {
12        throw new NotImplementedException("Not implemented");
13    }
14
15    private Vacuna[] constan;
16
17    private Mascota tiene;
18
19 }
```

Código 6: Clase CarnetVacunacion.

```
1 using System;
2
3 public class Vacuna {
4     private int idVacuna;
5     private string nombre;
6     private DateTime fechaAplicacion;
7     private DateTime proximaDosis;
8     private string lote;
9
10    public bool Registrar() {
11        throw new System.NotImplementedException("Not implemented");
12    }
13
14    private CarnetVacunacion constan;
15
16 }
```

Código 7: Clase Vacuna.

```
1 using System;
2
3 public class ValidacionMedica {
4     private int idValidacion;
5     private EstadoValidacion estado;
6     private DateTime fechaRevision;
7     private string observacionesRevisor;
8
9     public void IniciarRevision() {
10        throw new System.NotImplementedException("Not implemented");
11    }
12    public bool Validar() {
13        throw new System.NotImplementedException("Not implemented");
14    }
15    public void Rechazar(string motivo) {
16        throw new System.NotImplementedException("Not implemented");
17    }
18
19    private Usuario revisa;
20
21    private Mascota pasa;
22
23 }
```

Código 8: Clase ValidacionMedica.

```
1 using System;
2
3 public class AntecedenteGenetico {
4     private int idAntecedente;
5     private string descripcion;
6     private string parentesco;
7     private string condicionHereditaria;
8
9     public bool Registrar() {
10        throw new System.NotImplementedException("Not implemented");
11    }
12    public string Consultar() {
13        throw new System.NotImplementedException("Not implemented");
14    }
15
16    private Mascota posee;
17
18 }
```

Código 9: Clase AntecedenteGenetico.

```
1 using System;
2
3 public enum TipoUsuario {
4     PROPIETARIO,
5     REFUGIO,
6     VETERINARIA,
7     ADOPTANTE,
8
9 }
```

Código 10: Clase TipoUsuario.

```
1 using System;
2
3 public enum EstadoValidacion {
4     PENDIENTE,
5     EN_REVISION,
6     VALIDADO,
7     RECHAZADO,
8
9 }
```

Código 11: Clase EstadoValidacion.

Instrucciones detalladas de reproducibilidad

Para reproducir el ciclo completo a partir del repositorio:

1. Clonar el repositorio: `git clone https://github.com/MiloSaurio4kHD/PFC-MundiPets-MDD-EquipoG.git`
2. Abrir el modelo `*modelo/MundiPets-DiagramaClases.vpp` en Visual Paradigm 17.3 o superior.
3. Ejecutar la generación desde `Tools → Code → Instant Generator`, seleccionando el lenguaje `C#`, marcando las ocho clases y las dos enumeraciones, y estableciendo como salida la carpeta `generado/`, con la configuración documentada en la Sección 5.1 y en `plantillas/`.
4. Abrir los archivos `.cs` resultantes en un entorno de `C#` (Visual Studio Community o superior con `.NET SDK`) para compilarlos y ejecutar la unidad demostrable.
5. Para compilar el informe: en la carpeta `docs/`, ejecutar `pdflatex informe.tex`, luego `biber informe`, y `pdflatex informe.tex` dos veces más.

Las versiones exactas de software y las dependencias se detallan en el archivo `README.md` de la raíz del repositorio, con el cual este anexo debe mantenerse consistente.